

MỤC LỤC

MỤC LỤC.....	1
TUẦN 4: React Framework.....	2
1. React State & Rendering Fundamentals.....	2
1.1. Khái niệm State trong React.....	2
1.2. Tính bất biến của State (State Immutability).....	2
1.3. State Lifting - chia sẻ logic & dữ liệu giữa các component.....	2
1.4. Khi nào component re-render? (cơ chế re-render trong React).....	3
2. React Hooks - Core Hooks Group (use)*.....	4
2.1. useState.....	4
2.2. useEffect.....	4
2.3. useMemo.....	6
2.4. useCallback.....	7
2.5. useRef.....	8
2.6. useContext.....	9
3. React Rendering Architecture.....	10
3.1. ReactDOM.createRoot().....	10
3.2. Component Tree.....	10
3.3. Sự khác nhau giữa Root và Component Tree.....	10
4. Component Architecture & Patterns.....	11
4.1. Component Composition.....	11
4.2. Controlled Components.....	11
4.3. Uncontrolled Components.....	11

TUẦN 4: React Framework

1. React State & Rendering Fundamentals

1.1. Khái niệm State trong React

Trong React, *state* là dữ liệu năng động dùng để quyết định giao diện sẽ hiển thị như thế nào tại từng thời điểm. Khác với props - vốn được truyền từ component cha xuống và không thể thay đổi ở component con - state là dữ liệu nội tại, thuộc quyền quản lý của từng component. Khi state thay đổi, React sẽ chủ động cập nhật lại phần giao diện liên quan, giúp UI luôn đồng bộ với trạng thái ứng dụng. Đây là một trong những cơ chế cốt lõi của React, đảm bảo việc xây dựng ứng dụng trở nên trực quan, dễ bảo trì và phản hồi tốt với hành động của người dùng.

1.2. Tính bất biến của State (State Immutability)

React yêu cầu state phải được giữ dưới dạng *bất biến*, nghĩa là không được thay đổi trực tiếp bằng các thao tác như gán lại giá trị hoặc chỉnh sửa mảng và object tại chỗ. Thay vào đó, ta phải luôn tạo một bản sao mới của dữ liệu rồi cập nhật thông qua hàm setter. Tính bất biến giúp React nhận ra rằng state đã thay đổi và kích hoạt quá trình re-render một cách chính xác. Nếu state bị thay đổi trực tiếp, React sẽ không phát hiện được sự thay đổi đó dẫn đến UI không được cập nhật đúng. Ngoài ra, immutability còn giúp tránh lỗi khó đoán, tăng tính minh bạch của logic và hỗ trợ tối ưu hóa như memoization trong các hook nâng cao.

1.3. State Lifting - chia sẻ logic & dữ liệu giữa các component

Trong React, dữ liệu luôn chảy theo một chiều từ component cha xuống component con thông qua props. Tuy nhiên, khi nhiều component con cần truy cập hoặc thay đổi cùng một dữ liệu, việc đặt state ở từng component riêng lẻ sẽ gây dư thừa và khó đồng bộ. Để giải quyết vấn đề này, React sử dụng kỹ thuật *state lifting*, nghĩa là đưa state lên một component cha chung, sau đó truyền dữ liệu và hàm cập nhật xuống các component con. Cách tổ chức này đảm bảo mọi component đều làm việc với cùng một nguồn dữ liệu trung tâm, giúp tránh sai lệch và giữ cho logic ứng

dụng rõ ràng. State lifting là nền tảng quan trọng trước khi tiếp cận các giải pháp quản lý state phức tạp hơn như Context API hoặc các thư viện state management.

1.4. Khi nào component re-render? (cơ chế re-render trong React)

React sẽ tiến hành re-render một component khi một trong ba yếu tố sau thay đổi: state nội bộ của component, props được truyền vào từ component cha, hoặc context mà component đang sử dụng. Khi một thay đổi xảy ra, React tạo ra Virtual DOM mới, so sánh với bản cũ và xác định những khác biệt cần phải cập nhật vào DOM thật. Quá trình này giúp React chỉ cập nhật đúng phần giao diện thay đổi thay vì vẽ lại toàn bộ, giữ hiệu năng ổn định. Điều quan trọng là không phải bất kỳ tác động nào cũng khiến component re-render; chỉ những thay đổi được React nhận diện là khác biệt so với giá trị trước đó mới kích hoạt quá trình này.

2. React Hooks - Core Hooks Group (use)*

2.1. useState

2.1.1. Quản lý state trong function component

Trong React hiện đại, useState là hook cơ bản nhất giúp quản lý dữ liệu thay đổi theo thời gian trong function component. Khi gọi useState, React tạo ra một cặp gồm giá trị state hiện tại và một hàm setter dùng để cập nhật giá trị đó. Mỗi function component có thể sử dụng nhiều state khác nhau, và mỗi lần gọi useState là một “ô nhớ” riêng biệt được React theo dõi. Việc cập nhật state phải thông qua setter để React có thể biết rằng dữ liệu đã thay đổi. Nhờ cơ chế này, function component vừa ngắn gọn vừa dễ hiểu, thay thế hoàn toàn nhu cầu dùng class component để quản lý state như trước đây. useState là nền tảng của mọi logic động trong React, từ cập nhật input đơn giản cho đến quản lý dữ liệu phức tạp.

2.1.2. Re-render khi state thay đổi

Cơ chế re-render của React phụ thuộc trực tiếp vào state. Khi state được cập nhật bằng setter, React sẽ đánh dấu component cần re-render và tạo lại Virtual DOM để so sánh với bản cũ trước khi cập nhật phần DOM thật thay đổi. React chỉ re-render khi giá trị state mới khác giá trị trước đó, nên nếu setter nhận cùng một giá trị, quá trình render sẽ được bỏ qua. Bên cạnh đó, khi state của component cha thay đổi, các component con cũng có thể re-render nếu chúng nhận props liên quan đến state đó.

2.2. useEffect

useEffect là hook dùng để xử lý các tác vụ nằm ngoài quá trình render UI, thường được gọi là side effects. Những tác vụ này bao gồm gọi API, thao tác với DOM, xử lý sự kiện toàn cục hoặc thiết lập kết nối với các dịch vụ bên ngoài. useEffect cho phép React chạy một đoạn logic sau khi render hoàn tất, giúp tách biệt rõ ràng giữa quá trình tính toán giao diện và các hành động hỗ trợ của ứng dụng.

2.2.1. Dependency Array

Dependency array là cơ chế giúp React kiểm soát thời điểm một effect được thực thi. Bằng cách quan sát sự thay đổi của các giá trị trong mảng phụ thuộc, React quyết định nên chạy effect bao nhiêu lần và khi nào.

- Array rỗng [] : Khi dependency array rỗng, effect chỉ chạy một lần sau lần render đầu tiên của component. Trường hợp này thường dùng cho các tác vụ cần khởi tạo ban đầu như gọi API đầu trang hoặc đăng ký các event listeners một lần duy nhất.
- Có giá trị trong array: Khi mảng phụ thuộc chứa một hoặc nhiều giá trị, effect sẽ chạy lại mỗi khi một trong các giá trị đó thay đổi. Đây là cách thông dụng để đồng bộ dữ liệu, theo dõi thay đổi của props hoặc state cụ thể, và cập nhật UI hoặc dữ liệu bên ngoài tương ứng.
- Không truyền dependency array: Nếu không truyền dependency array, effect sẽ chạy sau mỗi lần render của component. Điều này đồng nghĩa bất kỳ sự thay đổi state hoặc props nào cũng làm effect thực thi lại. Cách dùng này phù hợp cho các tác vụ cần đồng bộ hóa liên tục với mỗi lần render, nhưng nếu không cẩn thận có thể gây ra vòng lặp hoặc tốn tài nguyên.

2.2.2. Cleanup Function

Cleanup function trong useEffect là đoạn mã chạy trước khi effect được thực thi lại hoặc trước khi component bị tháo khỏi giao diện. Cleanup thường dùng để hủy đăng ký sự kiện, dừng interval hoặc timeout, đóng kết nối hoặc giải phóng tài nguyên. Việc sử dụng cleanup giúp tránh rò rỉ bộ nhớ và đảm bảo component hoạt động ổn định ngay cả khi re-render nhiều lần. React luôn đảm bảo cleanup được gọi đúng thời điểm, tạo nên vòng đời rõ ràng cho mỗi effect.

2.2.3. Các pattern thường gặp

- Fetching data: useEffect thường được dùng để gọi API sau khi component render. Bằng cách dùng dependency array rỗng, ta đảm bảo API chỉ được gọi một lần. Đây là pattern phổ biến nhất, đặc biệt trong các ứng dụng lấy dữ liệu từ server.

- Subscriptions: Khi một component cần lắng nghe sự kiện từ nguồn bên ngoài như WebSocket, Firebase hoặc store tùy chỉnh, effect sẽ thiết lập subscription khi chạy, và cleanup function sẽ hủy đăng ký khi component bị gỡ hoặc khi thông tin phụ thuộc thay đổi. Điều này đảm bảo component luôn nhận đúng dữ liệu mới và không gây trùng lặp listener.
- Event listeners: useEffect cũng được dùng để gắn các event listeners lên window hoặc document, ví dụ như lắng nghe scroll hoặc resize. Cleanup function sẽ đảm nhiệm việc gỡ listener, giúp ứng dụng tránh lỗi trùng event và cải thiện hiệu năng.

2.3. useMemo

useMemo là hook được React cung cấp để tối ưu hiệu năng bằng cách ghi nhớ kết quả của một phép tính nhằm tránh chạy lại những logic không cần thiết. Trong những trường hợp component có các tính toán tốn kém như xử lý dữ liệu lớn, lọc danh sách phức tạp hoặc tạo ra các cấu trúc dữ liệu mới trong mỗi lần render, useMemo giúp giảm tải bằng cách chỉ tính toán lại khi thật sự cần thiết. Cơ chế của useMemo dựa trên sự thay đổi của các giá trị phụ thuộc, cho phép kiểm soát chính xác thời điểm kết quả được tái sử dụng hoặc cập nhật.

2.3.1. Memoization & tối ưu hiệu năng

Memoization là kỹ thuật lưu lại kết quả của một hàm để tránh phải tính toán lại nếu các đầu vào không thay đổi. useMemo áp dụng kỹ thuật này trong React để tối ưu quá trình render. Mỗi khi component re-render, React sẽ kiểm tra các giá trị phụ thuộc mà useMemo theo dõi. Nếu không có sự thay đổi, React sẽ dùng lại kết quả đã tính trước đó thay vì chạy lại hàm. Điều này đặc biệt hiệu quả trong các tình huống cần xử lý dữ liệu nặng hoặc khi việc tạo ra object hay array mới khiến các component con không cần thiết phải re-render. Nhờ useMemo, ứng dụng hoạt động mượt mà hơn và giảm được các phép tính dư thừa.

2.3.2. Dependency array trong useMemo

Dependency array là trung tâm vận hành của `useMemo`. Nó xác định khi nào React cần tính toán lại giá trị mới. Nếu một giá trị trong dependency array thay đổi, hàm bên trong `useMemo` sẽ được chạy lại để cập nhật kết quả. Ngược lại, nếu tất cả giá trị phụ thuộc giữ nguyên, React sẽ trả về giá trị memoized của lần trước. Cơ chế này giống với dependency array của `useEffect` nhưng thay vì chạy side effect, `useMemo` chỉ tập trung vào việc trả về giá trị tối ưu hóa. Việc xác định đúng dependencies là rất quan trọng; thiếu phụ thuộc có thể gây ra hành vi sai lệch, trong khi dư phụ thuộc có thể khiến `useMemo` mất hiệu quả.

2.3.3. Khi nào dùng và khi không nên dùng

`useMemo` nên được dùng khi giá trị cần tính toán phức tạp hoặc khi việc tạo lại dữ liệu mới có thể gây ra re-render không mong muốn ở các component con. Những tình huống như lọc danh sách lớn, thực hiện phép tính nặng hoặc truyền object/array xuống component con là ví dụ điển hình. Tuy nhiên `useMemo` không nên được sử dụng quá mức. Nếu giá trị được tính đơn giản hoặc render không gây tốn kém, việc memoization chỉ khiến code phức tạp hơn mà không mang lại lợi ích thực tế. `useMemo` là một công cụ tối ưu hóa, và tối ưu hóa chỉ thực sự cần thiết khi đã xác định được hiệu năng là vấn đề cần giải quyết.

2.4. `useCallback`

`useCallback` là hook giúp tối ưu hiệu năng bằng cách ghi nhớ (memoize) chính hàm được khai báo trong component. Trong React, mỗi lần component re-render, mọi hàm bên trong đều được tạo lại. Điều này có thể gây re-render không cần thiết ở component con nếu chúng phụ thuộc vào tham chiếu hàm. `useCallback` giúp ổn định tham chiếu bằng cách chỉ tạo hàm mới khi các giá trị phụ thuộc thay đổi.

2.4.1. Memoize function

`useCallback` lưu lại phiên bản của hàm và chỉ tái tạo khi dependency array thay đổi. Ở các lần render tiếp theo, nếu dependencies không đổi, React trả về chính hàm đã memoized thay vì tạo hàm mới. Điều này rất quan trọng khi truyền hàm xuống component con, vì sự thay đổi tham chiếu dù logic giống nhau vẫn khiến React hiểu rằng props đã thay đổi.

2.4.2. Kết hợp với React.memo

React.memo giúp component con chỉ re-render khi props thay đổi. Tuy nhiên, nếu không có useCallback, mỗi lần render của component cha sẽ tạo ra một hàm mới, khiến React.memo bỏ qua tối ưu hóa. Khi dùng useCallback, tham chiếu của hàm được giữ cố định, giúp React.memo nhận biết đúng khi nào props thật sự thay đổi. Sự kết hợp này giúp giảm đáng kể các lần re-render không cần thiết và là cách tối ưu phổ biến trong ứng dụng React quy mô lớn.

2.5. useRef

useRef là hook cho phép tạo ra một giá trị có thể thay đổi mà không làm component re-render. Khác với state, ref tồn tại xuyên suốt vòng đời component và giữ nguyên tham chiếu giữa các lần render. Điều này giúp useRef trở thành công cụ hữu ích khi cần lưu trữ dữ liệu mang tính ổn định hoặc thao tác trực tiếp với DOM mà không ảnh hưởng đến quá trình render của React.

2.5.1. Mutable Reference

useRef tạo ra một object với thuộc tính current có thể thay đổi mà không làm component re-render. Nhờ tính chất này, ref hoạt động giống như một biến lưu trữ lâu dài trong vòng đời component, phù hợp cho các giá trị tạm thời hoặc dữ liệu không ảnh hưởng đến UI.

2.5.2. Truy cập trực tiếp DOM

useRef thường được dùng để tham chiếu tới phần tử DOM. Khi gán ref vào một element, React sẽ đặt DOM thật vào ref.current, cho phép thực hiện các thao tác như focus input, cuộn, hoặc đọc kích thước phần tử. Đây là cách thao tác DOM đúng chuẩn trong React thay vì dùng query selector.

2.5.3. Lưu giá trị trước đó

useRef cho phép lưu lại giá trị ở lần render trước mà không gây re-render. Bằng cách cập nhật ref.current sau mỗi render, ta có thể so sánh giá trị cũ và mới để xử lý logic theo dõi thay đổi mà không cần tạo thêm state.

2.6. useContext

useContext là hook giúp component truy cập dữ liệu được chia sẻ thông qua Context mà không cần truyền props theo từng cấp component. Thay vì phụ thuộc vào luồng dữ liệu từ cha xuống con, useContext cho phép truy cập trực tiếp vào nguồn dữ liệu chung, giúp cấu trúc component trở nên gọn gàng và dễ bảo trì hơn. Đây là công cụ quan trọng khi nhiều component cần cùng một thông tin mà không muốn truyền props lồng nhau.

2.6.1. Context API & Provider

Context API tạo ra một vùng dữ liệu chia sẻ. Dữ liệu này được cung cấp bởi Provider, và mọi component bên trong Provider có thể lấy dữ liệu đó thông qua useContext. Cách tiếp cận này giúp quản lý dữ liệu tập trung và hạn chế việc truyền props thừa thãi.

2.6.2. Tránh Prop Drilling

useContext giúp loại bỏ prop drilling, tức việc phải truyền props qua nhiều component trung gian dù chúng không dùng dữ liệu. Component cần dữ liệu có thể truy cập trực tiếp từ Context, giúp cấu trúc gọn và dễ bảo trì hơn.

2.6.3. Khi nào nên dùng Context

Context phù hợp cho dữ liệu mang tính toàn cục như thông tin người dùng, theme, hoặc cài đặt ứng dụng. Tuy nhiên, không nên dùng Context cho state thay đổi liên tục vì có thể gây re-render rộng. Với các trường hợp đơn giản, lifting state vẫn hiệu quả hơn.

3. React Rendering Architecture

React Tree Root: React Tree Root là điểm khởi đầu của toàn bộ ứng dụng React trong trình duyệt. Từ root này, React xây dựng và quản lý toàn bộ cây component. Đây là nơi kết nối giữa React và DOM thật, giúp React có quyền kiểm soát việc render, cập nhật và gỡ bỏ các phần tử trong giao diện.

3.1. ReactDOM.createRoot()

ReactDOM.createRoot() khởi tạo ứng dụng React theo cơ chế concurrent mới. Sau khi tạo root, phương thức .render() sẽ gắn cây component vào phần tử DOM. Đây là bước React thiết lập môi trường trước khi UI được hiển thị.

3.2. Component Tree

Component Tree là cấu trúc phân cấp của tất cả component trong ứng dụng. React dùng Virtual DOM để so sánh và chỉ cập nhật những phần thay đổi, giúp render hiệu quả mà không phải vẽ lại toàn bộ giao diện.

3.3. Sự khác nhau giữa Root và Component Tree

Root là điểm gắn kết giữa React và DOM thật, còn Component Tree là toàn bộ cấu trúc UI bên trong root. Root hầu như cố định, còn Component Tree thay đổi liên tục khi state và props cập nhật.

4. Component Architecture & Patterns

4.1. Component Composition

Component Composition là cách kết hợp nhiều component nhỏ để tạo nên một component lớn hơn. Thay vì xây dựng một component chứa nhiều chức năng, React khuyến khích tách chúng thành các phần độc lập rồi ghép lại thông qua props hoặc children. Cách tiếp cận này giúp component dễ tái sử dụng, dễ kiểm thử và giảm độ phức tạp của UI. Composition được xem như “trái tim” của React, vì nó cho phép xây dựng giao diện theo dạng kết hợp các khối chức năng đơn giản.

4.2. Controlled Components

Controlled Components là các component có giá trị của input được điều khiển hoàn toàn bởi state trong React. Mỗi lần người dùng nhập dữ liệu, giá trị được gửi vào handler để cập nhật state, và state lại điều khiển UI. Điều này đảm bảo dữ liệu luôn đồng bộ giữa state và giao diện, phù hợp cho các form cần kiểm soát chặt chẽ như validation, xử lý realtime hoặc tạo trải nghiệm đồng bộ xuyên suốt.

4.3. Uncontrolled Components

Uncontrolled Components là các component mà giá trị input được quản lý trực tiếp bởi DOM thay vì React. Thay vì state, ta lắng nghe dữ liệu bằng ref khi cần lấy giá trị. Uncontrolled Components đơn giản hơn, ít code hơn và phù hợp cho các form không yêu cầu kiểm soát liên tục. Tuy nhiên, vì React không theo dõi giá trị đầu vào theo thời gian thực, pattern này kém linh hoạt hơn trong các trường hợp cần validation hoặc xử lý logic phức tạp.